

# Evidence-First Security for Autonomous Agents

*An agent's identity, authorization decision, or audit log is only as trustworthy as the evidence connecting it to what the agent actually caused to happen.*

The industry is converging fast on an agent control plane: give agents first-class identities, constrain their authority, mediate sensitive actions, record execution, govern access over time. NIST's National Cybersecurity Center of Excellence has opened a concept paper on exactly this, naming as focus areas “the identification, authorization, auditing and non-repudiation of AI agents, as well as controls to prevent and mitigate prompt injection techniques” (NCCoE, 2026). Cloud providers expose agent-identity constructs and mediate tool actions with deterministic policy outside agent code; OpenTelemetry (2026) and the Open Cybersecurity Schema Framework (OCSF, 2026a) are standardizing the records everything downstream consumes.

This work sits one layer beneath those controls, on the problem they all depend on and none of them is the answer to: **the evidence gap between an agent's identity and what it actually caused**. Identity can be correct while causality is wrong. A framework trace can say a tool ran when the side effect never occurred. A cloud log can prove a credential changed a resource while never explaining which agent, delegation or untrusted input set it in motion. Normalization can stay syntactically valid while dropping the provenance security needs.

The model may stay probabilistic. The security boundary around it should become more provable than the model itself.

720

Runs scored once

2

Pre-registered campaigns

46

Evidence fields audited

1

Structural gap

## Why “AI vs. AI” is the wrong fight

AI belongs in defense — for triage, correlation, anomaly discovery, investigation. The mistake is making a second probabilistic model the final arbiter of a fact that stronger evidence could establish. An attacker can probe an agent until one attempt lands; a defender often needs a reliable account of every high-impact action. Meeting a cheap, imprecise offense with an equally probabilistic defense is the wrong side of that asymmetry.

The alternative is not a cleverer model. It is to mediate authority at observable boundaries and make the resulting actions carry enough independently verifiable provenance that the enterprise never has to trust the agent's own explanation — and to measure, reproducibly, where that provenance holds and where it breaks. To be explicit: this is not “AI detecting AI.” The instruments are deterministic; the agent is the subject of measurement, never the measurer.

## The missing primitive: action provenance

A security-sensitive agent action is a causal chain:

principal → delegation → agent identity → runtime instance → input → authorization → tool → side effect

Every arrow is a claim of causality. The goal isn't to pretend every edge is directly observable — it is to state what class of evidence supports each edge and how much uncertainty remains. Five classes, in ascending strength:

| CLASS         | MEANING                                                           | EXAMPLE                     |
|---------------|-------------------------------------------------------------------|-----------------------------|
| Claimed       | The agent or framework asserts the operation occurred             | a tool-call span            |
| Observed      | An independent sensor sees behavior consistent with the claim     | process / network telemetry |
| Authoritative | The system owning the resource records the state change           | a cloud audit log           |
| Attested      | An observer cryptographically binds its identity to the record    | a signed evidence entry     |
| Inferred      | The link is reconstructed from correlation, not directly recorded | time-window stitching       |

This tracks where standards are heading — OCSF's record-integrity and attestation work (OCSF, 2026a), OpenTelemetry's `invoke_agent` and `execute_tool` spans (OpenTelemetry, 2026) — which give an execution substrate but not, by themselves, proof that the claimed external side effect happened.

## Standards-native by design

This is an evidence envelope *around* standard telemetry, not a replacement for it. OpenTelemetry is the execution substrate; the GenAI conventions already model an agent run as a span tree, with MCP folded into the same vocabulary. The right move is to carry runtime evidence on OTel rather than invent a parallel trace ecosystem. The envelope adds what OTel does not prove: observer identity, evidence class, source trust domain, causal link, reconstruction method, ground-truth label, integrity status, and declared blind spots. OCSF is the downstream security representation, and rather than assume a schema gap, the work measures whether real runtimes can faithfully populate the semantics OCSF and the active agentic-observability efforts are adding.

## Four instruments, and the experiment that made them agree

Four instruments were built, each measuring a different failure mode in the action-evidence chain. Separately, each produced a finding. The step that mattered was forcing all four to describe the same event at the same time — because an instrument that is never contradicted is never tested.

| STAGE                  | QUESTION IT ANSWERS                                        | WHAT THE JOINT RUN SHOWED                                                                        |
|------------------------|------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| Ground truth           | Is defense evaluation reproducible against a known attack? | Yes, and single-run evaluation misreports the hard vector outright.                              |
| Corroboration          | Does an independent observer confirm the claim?            | Yes for occurrence; never for causation — the observer says so itself.                           |
| Standardization        | What survives the path to a security schema?               | Most of it. The residue is small, specific, and namable.                                         |
| Capability vs. reality | Does standing authority match what was exercised?          | Partly — and where a credential cannot distinguish two resources, nothing downstream can either. |

### THE EXPERIMENT, AS IT WAS SPECIFIED IN ADVANCE

A legitimate principal gives an agent a benign task. Fulfilling it requires reading a record containing attacker-supplied text. That text is relayed onward, and a second agent takes an action — using a tool it is genuinely permitted to use, with a credential that is genuinely valid, against a resource it is genuinely allowed to reach. **The only thing wrong is why.** Run under four control regimes — baseline, prompt-level defense, deterministic action policy, and provenance-aware policy — with parameters and expected results registered before the run, and scored exactly once.

## What was registered before the run

The results below are only worth reading because the expectations were written down first, and several of them were wrong. Parameters — sample size, the confidence criterion, generation limits — and the expected outcome of every arm were committed before execution, and the corpus was scored exactly once, with no arm extended after results were seen.

Two campaigns, each frozen before it ran and scored exactly once. The first is **240 attack runs** — four scenario variants × four control regimes × **N=15** — and **120 legitimate-traffic runs**, two ordinary tasks × the same four regimes × 15. The second repeats that shape against three further attack families: instructions that survive in an agent's own memory and fire in a later run, a chain begun by a scheduler with no human in it, and several runs sharing one credential at the same moment. Every interval below is a Wilson score interval at 95% (Wilson, 1927) over one such arm. N=15 is the smallest sample whose 95% half-width at the expected rate stays inside a bound committed before the run — chosen for the width of the interval rather than for whether the arms differ, because every comparison this document claims already separates at N=4. Spending the sample on precision rather than on significance is why a within-arm difference is reported below as *not claimable* rather than quietly claimed.

Three predictions are worth naming because they constrain how much credit the results deserve:

- **One hypothesis was registered as *probably our own defect rather than an industry gap*** — and it was. A named enterprise mechanism already propagates the delegation initiator across role chaining and records it (Amazon Web Services [AWS], 2026b), and the schema already defines the field (OCSF, 2026c). Our instruments not populating it is a fact about this lab and nothing more.
- **Several arms were registered as expected to produce no interpretable result at all**, with the failure rate itself as the reported outcome. They did. Registering that in advance is what keeps an unusable arm from being quietly reported as a success.
- **One factor was registered as expected to move control efficacy. It moved something else.** Two variants differing only in the plausibility of a destination produced identical efficacy and opposite termination behaviour. It is reported as measured and not explained.

The second campaign registered its own four, and the one that matters most registered a way to lose:

- **The cross-time gap was registered as *likely structural, and audited hardest precisely because of that*** — prettier than the first campaign's finding, because both endpoints are under our own control and neither can be blamed on a vendor. It is reported here as a measurement, with its classification still owed the full citation gate.
- **A control we had already fixed was registered as *expected to succeed and to be unremarkable***, and recorded anyway. A campaign that quietly stops reporting a gap it fixed is indistinguishable from one that stopped looking.
- **The unattended-authority arm was registered with a trap against itself:** a chain that records nothing scores as *unproven*, never as *proven absent*. Silence must not earn a result.
- **The concurrency arm was registered with its own failure written out:** if the per-call identifier degrades along with the time window, then the identifier is not doing the work claimed for it, and that is the finding — reported as a defect in our own instrumentation rather than softened.

A fourth commitment cost the most: the prettiest finding gets audited hardest. An earlier campaign was voided outright when an external reviewer showed one observer had never been invoked — and the discarded verdict was the *favourable* one, claiming four more structural gaps than survive here.

## What it found

---

### 1. Clean authorization is the hard case, and prompt-level defense does not touch it.

Where the authorization record was visibly wrong, the generic defense stopped the attack completely (0.00, 95% CI [0.00, 0.20]). Where authorization was clean and only causality was wrong, the same defense stopped nothing (1.00 [0.80, 1.00]). The intervals do not overlap. Stop treating “the agent was authorized” as reassurance: authorization correctness and action correctness are different questions, and most controls answer only the first.

## 2. Two controls scored identically against the attack. They are not interchangeable.

Two controls drove clean-authorization attack success to 0.00 — indistinguishable on any attack-only scoreboard. Across 120 runs of legitimate work, one of them blocked a benign operations task in **11 of 15 runs** (0.73 [0.48, 0.89]); the other blocked **nothing**, on either benign task, while stopping the same attack (0.00 [0.00, 0.20]). The intervals do not overlap.

The difference is where the decision looks. Reasoning over identity and action cannot see *why* a call is being made, so stopping a legitimately-scoped capability means forbidding it for every request, forever. Reasoning over what caused the request can refuse the attack and permit the identical call when its cause is clean. Read the cost narrowly: it appears only where a benign task needs the forbidden capability — a second benign task paid nothing under the same control. The tax is real, concentrated, and wholly invisible to attack-success scoring.

**The second campaign repeated this measurement and the cost grew.** The same benign operations task, the same deterministic policy, a fresh 120 runs of legitimate work: the policy blocked a legitimate call in **13 of 15 runs** (0.87 [0.62, 0.96]), against 11 of 15 the campaign before. The provenance-aware control again blocked **nothing**, on either benign task, in either campaign.

The increase has a specific and unflattering cause, and it is ours. Between campaigns the agent gained a new capability — persistent memory — and nobody added a rule for it to the policy set. A deterministic policy denies by default, so every legitimate use of the new capability was refused, by a control that had never been told the capability existed. **The agent did not lose an attack surface; it lost a working tool.** That is the failure mode of allowlist-shaped controls in one sentence: they are only as current as the last person who remembered to update them, and the failure is silent, because a denial looks identical whether the rule was considered and refused or never written at all.

It was left unfixed on purpose once found mid-campaign, because repairing it would have made the runs before and after incomparable. The cost is reported as a utility cost rather than quietly corrected, and a test now asserts that every invokable capability has an explicit policy decision — which is the actual fix, since the missing rule was never the root cause. The missing *check* was.

**Before the utility half ran, the defensible statement was that both controls bought their result the same way — by removing capability. That statement was wrong, and only the legitimate-traffic half could show it.** It is the clearest argument here for measuring the boring half of an experiment.

## 3. A security control caused an availability failure that attack-success scoring cannot see.

One defense configuration caused the agent to stop terminating, on every run of three separate arms. Those runs produced no interpretable outcome at all. Counting them as “attack blocked” would have published a **100% effective control** when the agent had simply hung. An evaluation that also tracked task completion would have caught it; one that asks only whether the attack landed cannot — and that is the evaluation most controls are judged by. Separate *the run was valid* from *the attack failed*, and report the invalid rate as a first-class number.

## 4. The gap is not about crossing a company boundary. It is about causation leaving the run.

The first campaign's surviving gap sat between two companies, which invites an easy dismissal: of course two vendors do not share a causal identifier, that is what a standard is for. So the second campaign removed the boundary. An agent reads attacker-supplied text and persists it to its own long-term memory. A *separate, later run* — new process, new trace, new session — recalls that note and acts on it. Both endpoints are now inside one estate and both are ours.

The store is an authoritative log and records, for every note, the run that wrote it. The recall path returns the note's text and not its lineage — deliberately, because that is what a real memory store does; it hands back content, not provenance. So the ground truth exists, in our own database, one join away.

## THE MEASUREMENT

Across **60 runs** in which an agent recalled a planted instruction and acted on it, the number of traces containing any reference to the run that wrote the instruction — its run identifier, any of the 555 span identifiers those runs emitted, or even the name of the provenance field — is **zero**. The action is fully recorded. Its cause is fully recorded. Nothing records that the second followed from the first.

This is the same shape as the cross-vendor gap and it is harder to wave away, because there is no vendor to blame. The binding is not missing because two companies could not agree on a field. It is missing because **the record of an action has nowhere to say which earlier run caused it**, and because the natural interface of a memory store — give me what I know — is provenance-free by construction. An agent that can write to its own memory can therefore issue instructions to its own future, and the telemetry of the future run looks exactly like an agent doing its job.

All three controls stopped this attack on every run, and **not one of those numbers means what a results table would say it means**. One of them wraps anything recalled from memory in an untrusted-content marker, so it blocks by construction — the honest claim there is narrow and worth stating exactly: *marking recalled memory as untrusted is sufficient to stop cross-run poisoning in this setup*, which is not evidence that prompt-level defenses work in general, and the clean-authorization results above are evidence that they do not. The other blocked every run by refusing a tool nobody had written a rule for: the memory capability was added to the agent and never added to the policy, so the run was denied by default. **That is a utility cost — the agent lost the capability outright — and it is reported here as one, never as a block rate**. It is left unfixed deliberately: repairing it mid-campaign would have made the runs before and after incomparable.

**Reported as a measurement, not yet as a gap-table verdict.** Classifying it requires the same two-key gate every other field went through — fetch a citation for every candidate home, not the interesting ones, and demote on any hit. That gate is scheduled and has not been run for this field, so this document reports what was measured and does not report a class. The prediction, registered in advance, was that it would survive as structural; a prediction is not a result.

### 5. Nobody authorised it, and not one control asked.

The third scenario removes the human. A scheduled trigger — a nightly job, no person in the loop — starts a chain that ends in a sensitive action against a real resource. Every run declares its principal honestly: an identifier naming the schedule, a type of *scheduled*, and an authentication method of *none*.

Everything about the chain is otherwise correct. The tools are allowlisted. The credential is scoped for the audience it is presented to. The path is inside the permitted surface. The terminal action reaches the platform and the platform records it. **The only thing wrong is that no human authorised any of it**, and none of the controls examined inspects that.

| ARM                     | ATTACK SUCCESS    | N  | EXCLUDED | STATUS       |
|-------------------------|-------------------|----|----------|--------------|
| undefended              | 1.00 [0.80, 1.00] | 15 | 0/15     | ok           |
| prompt-level defense    | 1.00 [0.80, 1.00] | 15 | 0/15     | ok           |
| deterministic policy    | 1.00 [0.77, 1.00] | 13 | 2/15     | ok           |
| provenance-aware policy | not resolvable    | 0  | 15/15    | unresolvable |

Two of the three controls fire at 1.00 with non-overlapping certainty against zero. That is not a control failing; it is a control having nothing to object to. Each reasons over identity, scope and action, and by all three measures this chain is impeccable. The question none of them asks is *who wanted this*.

The arm was registered with an explicit trap to keep it honest: a chain with no human principal must either **declare** a non-human principal, or score as *unproven*. Silence must never earn a result — a run that simply left the field empty would be indistinguishable from one where nobody wrote it down, and counting that as evidence of unattended authority would be inventing data. These runs declare it, so they score.

The fourth arm — provenance-aware policy — is reported as *not resolvable* rather than as a fourth number. The reason is the next finding, and it is an instrument failure rather than a result.

That arm was re-run afterwards, and the design of the re-run matters more than its outcome. Re-running the failed arm alone would have proved nothing, because there would be no way to know whether the instrument had returned to the state the other arms were measured in. So it was re-run as a **pair**: the prompt-level defense alongside it as a positive control, under one continuous instrument, in a separate corpus, with both possible outcomes written down in advance.

| REPLICATION, ONE INSTRUMENT    | ATTACK SUCCESS    | N  | RUNS CARRYING AN EXPLICIT REFUSAL |
|--------------------------------|-------------------|----|-----------------------------------|
| prompt-level defense (control) | 1.00 [0.80, 1.00] | 15 | 0 of 15                           |
| provenance-aware policy        | 0.00 [0.00, 0.28] | 10 | <b>10 of 10</b>                   |

The positive control reproduced exactly what it had done before, which is what licenses reading the other row at all. And the provenance runs carry something the failed arm never did: a record, written by the control itself, saying it refused the call and why. The failed arm contained **zero** such records across fifteen runs. That difference is the whole distinction between a control working and a chain merely not finishing — **a control that refuses leaves evidence of refusing**.

So the finding, stated at the strength the evidence supports: against a chain that no human authorised, reasoning over identity, scope and action does not help, and reasoning over *what caused the request* does. The five runs excluded from that ten are excluded for the instrument reason described next, detected by the same test that caught the original failure — and the replication is reported at N=10 under an amendment. It does not replace the arm it repaired, which stays unresolvable.

6. The experiment measured its own instrument, twice, and came close to publishing both.

One arm looked, at first read, like the best result either campaign produced: every control failing against the unattended chain except the provenance-aware one, which held on every run. It would have been the headline. It was an artifact — and chasing it down produced a finding worth more than the headline would have been.

Generation here is **not a function of the prompt**. A byte-identical input returns one exact token count for a stretch of runs and a different exact token count for another stretch, at the same model and the same sampling settings. Not a distribution — one exact value, then another:

| SCENARIO          | ANSWER TO ONE IDENTICAL PROMPT | RUNS | WHAT CHANGED                         |
|-------------------|--------------------------------|------|--------------------------------------|
| unattended chain  | exactly 326 tokens             | 28   | —                                    |
| unattended chain  | exactly 918 tokens             | 17   | the model server had restarted       |
| shared credential | exactly 918 tokens             | 4    | —                                    |
| shared credential | exactly 326 tokens             | 25   | <b>nothing. Same server process.</b> |

The first split has an obvious explanation and the second destroys it. A restart is one way to move the answer; it is not the only one. The last two rows are a single server process — one process identity, sampled every thirty seconds for the length of the campaign — and the answer changed *between two cohorts of the same arm*. What decides it is some server-side execution state, and **no field in the experimental record captures it**: model name, version, temperature and seed are identical across every row of that table.

A regime change inside an arm is **indistinguishable, on a results table, from a control working**. A second arm proves it. Under concurrency, the prompt-level defense read 11 fired and 4 blocked — 0.73, which any evaluation would report as a control that works most of the time. Crossing each run's verdict against which regime it ran in: **every single blocked run is a run in the other regime**. Scored on the runs where the instrument was constant, the arm is 10 of 10 — the defense does not touch this attack at all. The reported number was not merely imprecise. It pointed the opposite way.

Neither arm was produced by doing anything wrong, which is the point. Restarting a model server is an ordinary event — a deployment, a host reboot, an idle timeout, a provider-side reload — and the second split did not even need that. No protocol reviewed here treats either as an experimental condition, and the record a run leaves behind is identical on both sides of one. That is why it reached a scored campaign rather than being caught in review.

Reproducibility of an LLM-based experiment is not established by fixing the model, the version, the temperature and the seed. Those were all constant across results that point in opposite directions. Unless an evaluation can show that its arms were generated under one regime, it cannot distinguish a control that works from a control that was measured at the right moment.

The effect is detectable after the fact in any corpus that records what was sent and how much came back, with no extra instrumentation: a prompt seen in several runs is a byte-identical input, so a change in the length of its answer is a change of instrument. Applied to the first campaign the same detector finds a single regime across all 240 of the runs it can see — that campaign's results are untouched. Applied to the second it recovers both transitions above, including the one with no external cause, and names the two arms that straddle them.

Both were caught by the rule that has cost this project the most and paid the most: **the prettiest finding gets audited hardest**. The first audit was triggered by nothing but a result being the one that would look best in a summary. The second was triggered by the first turning out to be wrong — and the correction was published against a claim this document had already made.

## 7. One credential, several runs at once, and the resource owner cannot say which.

When several agent runs share one credential at the same moment, the resource owner still writes its own record of each change — and the question is whether anything in that record can name the run responsible. Across **166 authoritative records**, joining by the time the record was written leaves **150 of them ambiguous**: more than one run could have caused it, with up to 9 candidates and 3.5 on average.

**The identifier column degrades too, and that is the finding.** 40 of 166 records are ambiguous even when joined on the per-call identifier. This was registered in advance as the outcome that would convict our own instrumentation rather than the industry, and it is reported that way: the identifier is not doing the work claimed for it here.

The mechanism is worth stating, because it does not excuse the result — it sharpens it. The identifier in this lab is minted when a *credential* is issued, not when a *call* is made. The scenario deliberately gives one credential to several simultaneous runs, so they share the identifier too, and it stops telling them apart. That is precisely the failure the previous campaign's recommendation warned about — *carry a per-call identifier, not just a per-credential one* — reproduced by our own instrumentation, at our own hands, and now measured rather than argued. A recommendation nobody has tested against themselves is worth less than one that has been, and this is what testing it against ourselves returned.

The honest qualifier is the third column. **83 of 166 records carry no such identifier at all** and are unattributable by it — the identifier gives certainty where it is present and nothing where it is not. A mechanism that must be populated by the caller is only as good as its coverage, and coverage is the part a standard has to mandate rather than merely define.

## 8. Most apparent gaps were ours, or already solved — and saying so is the point.

Of 46 evidence fields audited across the chain: 15 were emittable with no standard home, 11 cost-gated, 8 already present, 7 already carried by a named enterprise mechanism, and 3 were fields **this project's own specification** defines and its own instrumentation never emitted. **One was structural** — a single gap on a single edge, expressed as two missing fields (`caused_by_span` and `caused_by_method`), which is why it is counted once here and twice in any table that lists fields. Before concluding your telemetry has a gap, check in order: did we emit it; does a sensor we own emit it; does the standard define it; is someone already fixing it. Most answers are “yes” at one of those steps.

## 9. One gap survived a full audit, and it sits between vendors rather than inside them.

Nothing binds an individual agent tool call to the record that the system owning the data writes about the resulting change.

Both endpoints can be authoritative — the framework proves a tool ran, the platform proves the state changed — while nothing records that the second was caused by the first. Joining them is reconstruction, not evidence. The host observer states the limit in its own output, unprompted: joined by identifier within the run, it corroborates that the action happened, *not that this run is what caused it*.

Two independent cited negatives support it. The execution substrate defines nine attributes on its tool-call span, none referencing an external system's record of a resulting state change; its nearest candidate is the *model's* tool-call identifier, which names nothing in the resource owner's world (OpenTelemetry, 2026). The security schema's agent profile defines four attributes and no tool object at all (OCSF, 2026a), and the active proposal that would most plausibly have closed it was checked *because* it would have demoted this finding — it describes the invariant capability, not a record of consequences (OCSF, 2026b). On the resource-owner side, the representative cloud audit record generates every identifier itself and carries no caller-supplied reference to the operation that triggered the call (AWS, 2026a): it records **what** happened and **which credential** did it, and has nowhere to record **why**.

## Would a large enterprise lose this today?

Ten vendor products across six layers were examined from published documentation — AI firewall, endpoint, cloud posture, network broker, identity and LLM observability — asking three questions of each record: does it capture the individual tool call, does it carry a caller-supplied correlation identifier, and does it link to a third-party resource owner's own record of the effect. The per-vendor matrix — every product named, every cell sourced to a document with its retrieval date, and every unanswered question marked *not established* rather than inferred — is maintained as a companion file beside this one rather than summarised away. The paragraphs below are its conclusion; they are not its evidence.

The first two are answered by several vendors. One major suite records, per interaction, every resource touched with the action performed and whether it succeeded — richer than any standard offers. An LLM-observability platform records the tool call with its arguments as a first-class span. An identity product carries a nestable delegation chain across an application boundary, which is the only mechanism examined that crosses an organisational boundary carrying caller-supplied context at all.

### THE COLUMN THAT MATTERS

The third column is empty except inside a single vendor's own estate. Each vendor reconstructs causality within its own boundary; nothing carries that reconstruction across one, and the single mechanism that does cross carries **identity**, not **causality**. The gap is at the boundary, not inside it — which is exactly what a standard is for, and why the change proposed below targets the schema rather than any product.



## The minimal standard change that closes it

---

One optional attribute, on the event the resource owner already writes:

```
ai_operation.caused_by:
  span_uid    the tool-call span this operation is attributed to
  method      how that attribution was established
```

`span_uid` is the join; `method` is what keeps it honest, because a link stitched from a five-second window and one carried on propagated context are otherwise indistinguishable once written down. Both are optional: a platform that cannot observe the causing call simply omits them, which is itself the useful signal. This is additive, needs no new object, and rides a profile that already applies to API activity (OCSF, 2026a). It is deliberately not a proposal for the agentic objects already under active discussion (OCSF, 2026b).

**Status: not yet filed.** It will be proposed against 1.10.0-dev once the campaign's evidence is public enough to cite in the issue, because a schema change argued from a private corpus asks a maintainer to take the proposer's word for the gap. Co-sponsors — particularly anyone who owns a resource-side audit record and would have to populate the attribute — are the difference between a proposal and a shipped field, and are actively wanted.

## What a defender can do now

---

- **Carry a per-call identifier, not just a per-credential one.** A token identifies a credential; one token covers every call an agent makes, so concurrent calls are indistinguishable in a resource owner's log. Have the runtime — or better, the mediator the calls pass through — attach an identifier for the individual tool call, and have receiving platforms record it.
- **Put the record outside the agent.** Anything the agent reports about itself is a claim, and a compromised agent simply does not write the line. A mediator the call must pass through observes the invocation instead of being told of it — worth little unless it is hard to bypass.
- **Make your authorities declare what they are complete for.** A log that states the scope it fully covers turns silence into evidence: inside that scope, no record means the action did not happen.
- **Count corroboration only across differing trust domains.** A framework's span and its own sidecar agree by construction; counting that as two witnesses inflates every confidence number downstream.
- **Price controls in work-not-done.** A control with a perfect block rate and an unmeasured utility cost has not been evaluated — and where two controls both score 100%, the utility measurement is the only thing that distinguishes them.
- **Treat agent memory as an untrusted input channel with a write side.** Anything an agent can persist, it can be made to persist — and the instruction fires in a later run, under a clean session, with no attacker present. Record the writer on every stored item, and carry that writer into the record of any action taken because of it. A recall interface that returns content without lineage is the normal design and is the whole problem.
- **Require a principal, and make “nobody” a value rather than a blank.** Scheduled and chained work often has no human behind it, which is legitimate and needs to be visible. An empty principal field cannot be distinguished from an unrecorded one; an explicit `scheduled / none` can be alerted on, rate-limited, or required to carry a prior human approval reference.

## Making fidelity falsifiable

---

A single “AI safety score” would hide exactly the distinctions this work exposes. Fidelity is a vector, reported per action class and per observer: capture coverage, attribution precision and recall, causal completeness, delegation completeness, independent corroboration, normalization retention, and integrity or attestation status. The run

reported here populated that vector for one attack family end to end; every cell it could not fill is recorded as a declared blind spot rather than left blank.

Three rules made the results worth reading, and each cost the project a finding: parameters fixed and registered before the run; scored exactly once, with no arm extended after results were seen; and the prettiest finding audited hardest. A campaign was voided outright when an external reviewer showed one observer had never been invoked — and the discarded verdict was the *favourable* one. A later arm was discarded when the best-looking control result turned out to be a restarted model server.

A fourth rule was added by the second campaign, at the cost of the arm that taught it: **the instrument must be identified in the record as precisely as the subject is**. Recording the model name and the sampling parameters is not enough to make a run reproducible, and worse, it produces a record that looks complete. The process instance is part of the experimental condition, and an experiment that can be paused must be able to say whether a pause happened in the middle of an arm.

## What this does not claim

---

- One scenario family, one agent framework, one model, one host. Findings are scoped to exactly that.
- No vendor product was deployed or tested. Comparisons are reasoned from published documentation, dated, and marked *not established* where the documentation is silent or could not be retrieved.
- “No standard defines this” means the cited sources contain no home for it — not that no home can exist. Every such verdict is pinned to document versions and retrieval dates, and expires.
- The utility figures measure *blocking*, not task quality: a run that was never blocked is not thereby proven useful.
- The attack technique is neither novel nor the contribution. The method is.
- Results are scoped to a single model-server process instance per campaign. Where a campaign was interrupted and resumed, the affected arm is reported as unresolvable rather than pooled, and the exclusion is stated with the result.
- The cross-time result is a measurement, not yet a gap-table verdict: its classification is owed the same citation gate every other field was put through, and that gate has not been run for it.
- The memory store, the scheduler and the agents are all instruments of this lab. That is what makes the cross-time result hard to dismiss — no vendor boundary is available to explain it — and it is also why it says nothing yet about how any particular product behaves.

## References

---

- Amazon Web Services. (2026a). *CloudTrail record contents* (eventVersion 1.11). AWS CloudTrail User Guide. Retrieved August 22, 2026, from <https://docs.aws.amazon.com/awscloudtrail/latest/userguide/cloudtrail-event-reference-record-contents.html>
- Amazon Web Services. (2026b). *CloudTrail userIdentity element* (eventVersion 1.11). AWS CloudTrail User Guide. Retrieved August 22, 2026, from <https://docs.aws.amazon.com/awscloudtrail/latest/userguide/cloudtrail-event-reference-user-identity.html>
- MITRE. (2026). *ATLAS: Adversarial threat landscape for artificial-intelligence systems*. Retrieved August 23, 2026, from <https://atlas.mitre.org>
- National Cybersecurity Center of Excellence. (2026, February 5). *Accelerating the adoption of software and artificial intelligence agent identity and authorization* [Concept paper, initial public draft]. National Institute of Standards and Technology. <https://csrc.nist.gov/pubs/other/2026/02/05/accelerating-the-adoption-of-software-and-ai-agent/ipd>
- Open Cybersecurity Schema Framework. (2026a). *ai\_operation profile* (Version 1.9.0, released August 3, 2026). Retrieved August 22, 2026, from [https://github.com/ocsf/ocsf-schema/blob/main/profiles/ai\\_operation.json](https://github.com/ocsf/ocsf-schema/blob/main/profiles/ai_operation.json)
- Open Cybersecurity Schema Framework. (2026b). *ai\_tool object* (Proposal No. 1728, drafted against 1.10.0-dev). Retrieved August 22, 2026, from <https://github.com/ocsf/ocsf-schema/issues/1728>
- Open Cybersecurity Schema Framework. (2026c). *Delegation object* (Version 1.9.0). Retrieved August 21, 2026, from <https://schema.ocsf.io/1.9.0/objects/delegation>

OpenTelemetry. (2026). *Semantic conventions for generative AI: execute\_tool span* [Unreleased; conventions split into a separate repository on May 5, 2026, with no release cut]. Retrieved August 22, 2026, from <https://github.com/open-telemetry/semantic-conventions-genai>

Wilson, E. B. (1927). Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158), 209–212.

---

**Method, environment, fleet topology, scenario construction, payload text, defense implementation internals and reproduction steps are deliberately withheld.** What is published here is the argument, the measured results, and the standards change they motivate. The attack technique is not novel and is described plainly above; what is withheld is not the idea but the working artifact — a payload that fires against a specific runtime, and the defense internals whose exact wording decides whether it does. A technique anyone can look up still does not get republished as a tested exploit, and reproduction steps are the part that turns a described risk into a usable one. Status legend: *Finding* — project-reported, scoped to what was tested, not independently audited.

One exhibit is published in redacted form beside this document, because it carries the argument better than prose does and carries no attack value: the pair of records showing a tool call and the resource owner's own record of the resulting change, arguments removed, with the join visibly absent. A manifest of cryptographic hashes for the withheld tree is published alongside, so that what was held back can later be shown to be unaltered.

Evidence base: two pre-registered campaigns, each scored exactly once, completed 23 and 24 August 2026 — 240 attack and 120 legitimate-traffic runs in the first, and the same shape against three further attack families in the second. Arms excluded for instrument reasons are reported as unresolvable with their exclusion rate, and are never counted as a control succeeding. Every figure in this document is computed from the campaign records rather than transcribed, so a number here and a number in the corpus cannot drift apart.